

OverTheWire: Bandit Wargame

A Beginner's Walkthrough — Level 0 to Level 15

By: *Janavika P. U., St. Joseph's College of Engineering*

This write-up documents my hands-on experience solving the first sixteen levels of the OverTheWire Bandit wargame. It includes the exact commands I used, screenshots of my terminal, the mistakes I made along the way, and what I learned from each one — written so that a complete beginner can follow along and actually understand what is happening, not just copy commands.

Author's note:

I've kept the mistakes in on purpose. Struggling and getting an error is a normal part of learning Linux and cybersecurity — seeing exactly where I went wrong and how I fixed it is often more useful than a perfectly clean run.

What is Bandit?

Bandit is a beginner-friendly "wargame" hosted by OverTheWire. It teaches the basic Linux command-line skills needed for cybersecurity (and general sysadmin work) through a series of levels. Each level gives you a username and a task; completing the task reveals a password for the next level's username.

You connect to each level using SSH:

```
ssh banditN@bandit.labs.overthewire.org -p 2220
```

Replace N with the level number. When prompted, paste in the password from the previous level. Level 0's password is public (it's "bandit0", posted on the Bandit website) — every password after that has to be found.

How to read this document

- Goal — what the level is actually asking you to do.
- Commands — exactly what I typed, explained in plain English.
- Screenshot — proof of the actual terminal session.

Mistake / Lesson — included wherever I fumbled, so you can avoid (or recognize) the same trap.

Level 0 → Level 1

Goal: Log in for the first time and find the password hidden in a file called "readme" in the home directory.

Commands used:

```
ls  
cat readme
```

"ls" lists whatever is in the current folder. It showed a single file called readme. "cat" prints a file's contents straight to the screen, so cat readme simply displayed the text inside it — and the password was written right there in plain English.

A terminal window with a black background and white text. The prompt is 'bandit0@bandit:~\$'. The user enters 'ls' and the output is 'readme'. The user then enters 'cat readme' and the output is a multi-line message: 'Enjoy your stay!', 'bandit0@bandit:~\$ ls', 'readme', 'bandit0@bandit:~\$ cat readme', 'Congratulations on your first steps into the bandit game!!', 'Please make sure you have read the rules at https://overthewire.org/rules/', 'If you are following a course, workshop, walkthrough or other educational activity,', 'please inform the instructor about the rules as well and encourage them to', 'contribute to the OverTheWire community so we can keep these games free!', and 'The password you are looking for is: 6y2kwnwK6grgvwvpvLaa2T1cpFEK0hNR'.

```
Enjoy your stay!  
bandit0@bandit:~$ ls  
readme  
bandit0@bandit:~$ cat readme  
Congratulations on your first steps into the bandit game!!  
Please make sure you have read the rules at https://overthewire.org/rules/  
If you are following a course, workshop, walkthrough or other educational activity,  
please inform the instructor about the rules as well and encourage them to  
contribute to the OverTheWire community so we can keep these games free!  
  
The password you are looking for is: 6y2kwnwK6grgvwvpvLaa2T1cpFEK0hNR
```

Fig 1 — Logging in as bandit0 and reading the password straight out of the readme file.

Lesson learned:

Always start a new level with "ls" to see what you're working with, before doing anything else.

Level 1 → Level 2

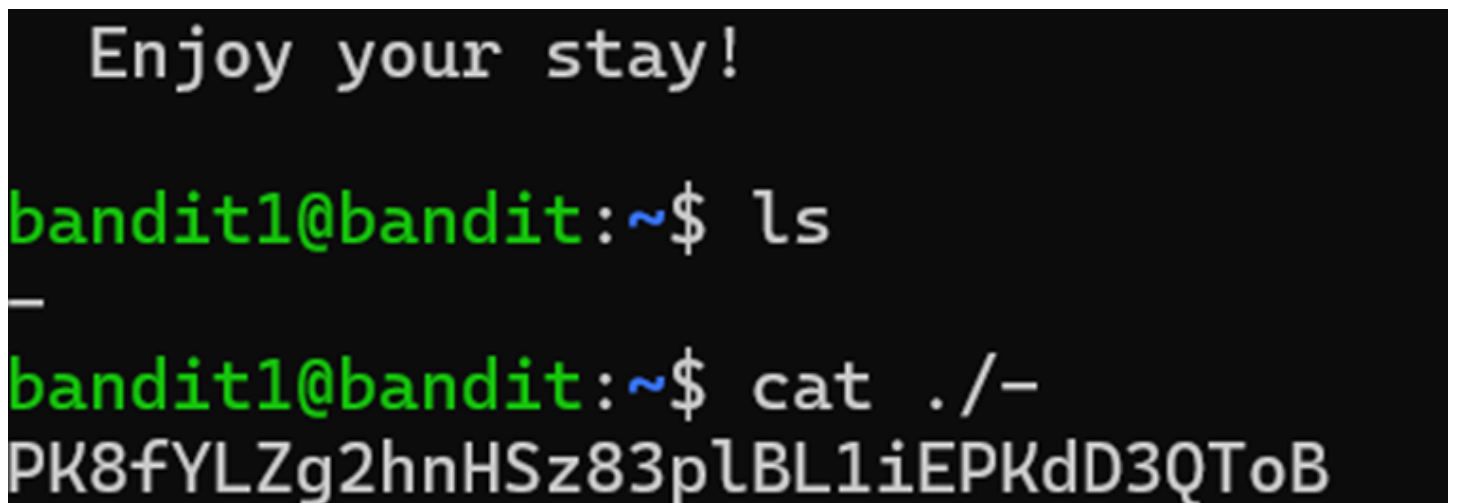
Goal: The password is in a file literally named "-" (a single dash) in the home directory.

Commands used:

```
ls
```

```
cat ./-
```

A filename of just "-" is a deliberate trap. If you type "cat -", the shell treats the dash as an option meaning "read from standard input" instead of treating it as a filename — so it just hangs, waiting for you to type something. The fix is to give the shell a proper path to the file so it can't be confused for an option: "./-" means "the file named - in the current directory."



```
bandit1@bandit:~$ ls
Enjoy your stay!
bandit1@bandit:~$ cat ./-
PK8fYlZg2hnHSz83p1BL1iEPKdD3QToB
```

Fig 2 — Using ./- instead of a bare - to read the file safely.

Lesson learned:

Any time a filename starts with a dash, prefix it with ./ (or its full path) so the shell doesn't mistake it for a command option.

Level 2 → Level 3

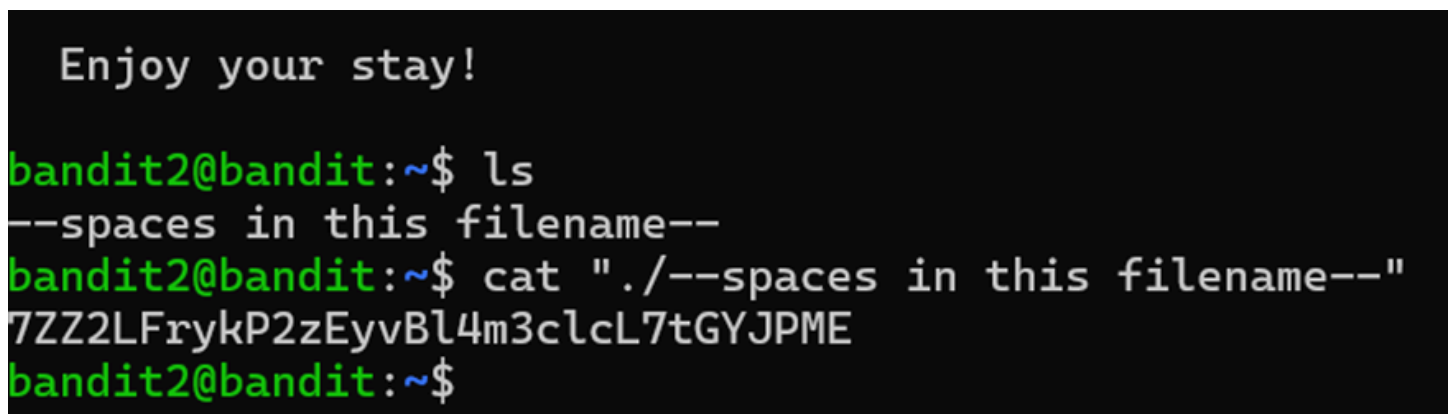
Goal: The password is in a file whose name contains spaces: "--spaces in this filename--".

Commands used:

```
ls
```

```
cat "--spaces in this filename--"
```

Normally the shell treats a space as a separator between arguments, so an unquoted filename with spaces gets split into several "words" and confuses the command. Wrapping the full filename in double quotes tells the shell "treat everything inside these quotes as one single argument."

A terminal window with a black background and green text. At the top, it says "Enjoy your stay!". Below that, the prompt "bandit2@bandit:~\$" is followed by the command "ls". The output is "--spaces in this filename--". Then, the prompt is followed by the command "cat "--spaces in this filename--"". The output is "7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME". Finally, the prompt is followed by a tilde and a dollar sign "~\$".

```
Enjoy your stay!  
bandit2@bandit:~$ ls  
--spaces in this filename--  
bandit2@bandit:~$ cat "--spaces in this filename--"  
7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME  
bandit2@bandit:~$
```

Fig 3 — Quoting a filename that contains spaces

Lesson learned:

Quote any filename that contains spaces or special characters — it's one of the most common everyday Linux gotchas.

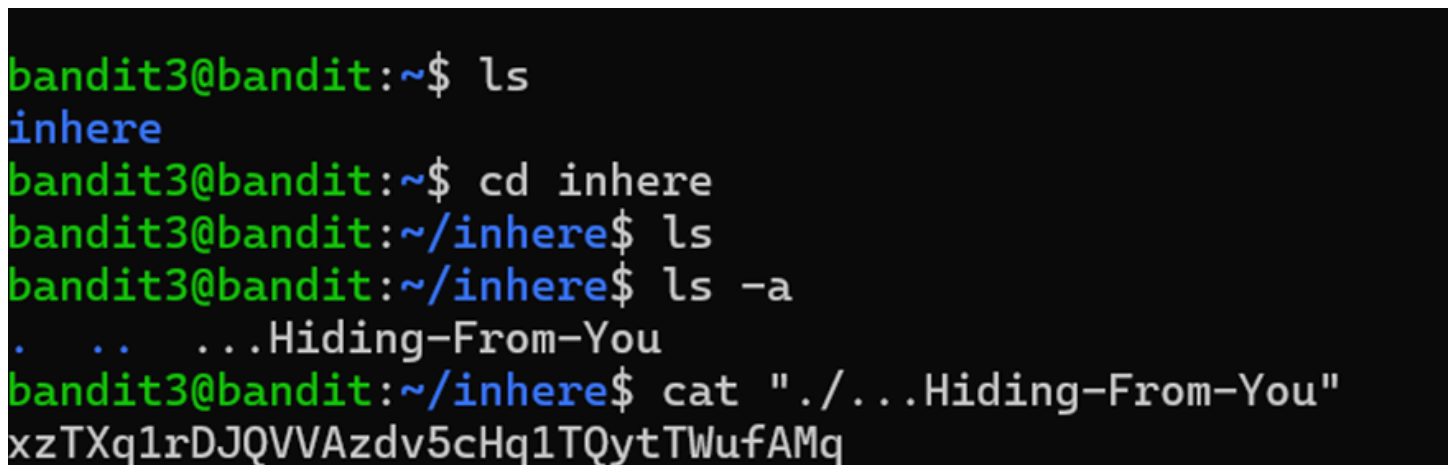
Level 3 → Level 4

Goal: The password is inside the inhere directory, in a hidden file.

Commands used:

```
cd inhere
ls
ls -a
cat "...Hiding-From-You"
```

A plain "ls" showed nothing, because in Linux any file or folder starting with a dot (.) is hidden by default. Adding the -a flag ("all") tells ls to show hidden entries too, which revealed a sneaky file literally named "...Hiding-From-You".

A terminal window with a black background and green text. The user is at the prompt 'bandit3@bandit:~\$'. They run 'ls' and see 'inhere'. They then run 'cd inhere'. At the prompt 'bandit3@bandit:~/inhere\$', they run 'ls' and see nothing. They then run 'ls -a' and see a hidden file named '...Hiding-From-You'. Finally, they run 'cat "...Hiding-From-You"' and the output is 'xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq'.

```
bandit3@bandit:~$ ls
inhere
bandit3@bandit:~$ cd inhere
bandit3@bandit:~/inhere$ ls
bandit3@bandit:~/inhere$ ls -a
.  ..  ...Hiding-From-You
bandit3@bandit:~/inhere$ cat "...Hiding-From-You"
xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq
```

Fig 4 – Revealing a hidden dotfile with ls -a.

Lesson learned:

If a folder looks empty, always double check with ls -a before assuming there's nothing there.

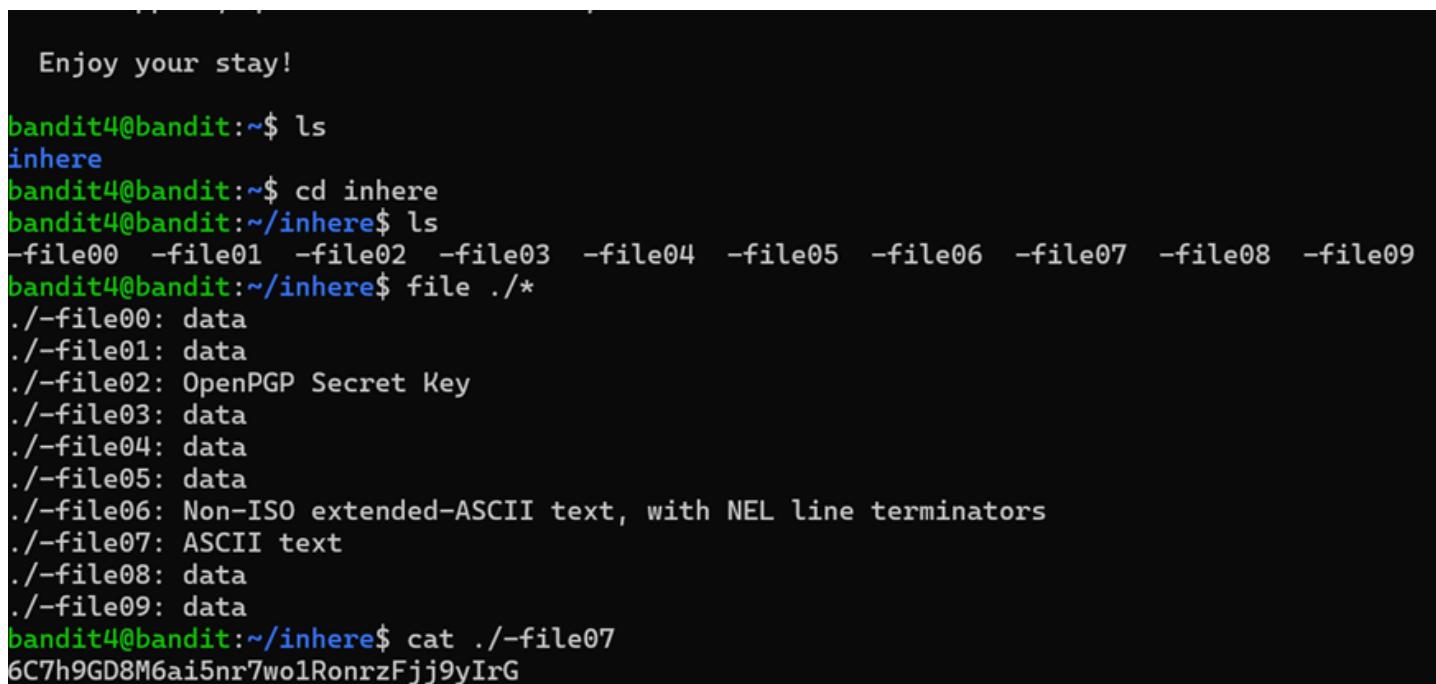
Level 4 → Level 5

Goal: The inhere directory has 10 files, only one of which is human-readable text; find and read it.

Commands used:

```
cd inhere
ls
file ./*
cat ./-file07
```

All ten files were named misleadingly (-file00 to -file09) and "ls" alone can't tell you what type of data is actually inside a file — only its name. The "file" command inspects each file's actual content and reports its real type. Running "file ./*" against all of them at once showed that only -file07 was plain ASCII text; the rest were binary data, an OpenPGP key, etc. Once identified, I cat'd only that one file.



```
Enjoy your stay!
bandit4@bandit:~$ ls
inhere
bandit4@bandit:~$ cd inhere
bandit4@bandit:~/inhere$ ls
-file00 -file01 -file02 -file03 -file04 -file05 -file06 -file07 -file08 -file09
bandit4@bandit:~/inhere$ file ./*
./-file00: data
./-file01: data
./-file02: OpenPGP Secret Key
./-file03: data
./-file04: data
./-file05: data
./-file06: Non-ISO extended-ASCII text, with NEL line terminators
./-file07: ASCII text
./-file08: data
./-file09: data
bandit4@bandit:~/inhere$ cat ./-file07
6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG
```

Fig 5 — Using file ./* to find which of ten similarly-named files is actually readable text.

Lesson learned:

Never trust a file's name or extension to tell you what it really is — the "file" command reads the actual content and tells you the truth.

Level 5 → Level 6

Goal: Somewhere inside inhere (which contains many nested folders), there is one file that is: human-readable, exactly 1033 bytes, and NOT executable.

Commands used:

```
cd inhere
```

```
ls
```

```
find . -type f -size 1033c ! -executable
```

With dozens of subfolders this is impossible to check by hand, so I used "find" to search recursively. -type f means "only regular files", -size 1033c means "exactly 1033 bytes" (c = bytes), and ! -executable excludes anything with the execute permission set. Find returned exactly one match, and cat'ing it gave the password.

```
Enjoy your stay!

bandit5@bandit:~$ ls
inhere
bandit5@bandit:~$ cd inhere
bandit5@bandit:~/inhere$ ls
maybehere00  maybehere03  maybehere06  maybehere09  maybehere12  maybehere15  maybehere18
maybehere01  maybehere04  maybehere07  maybehere10  maybehere13  maybehere16  maybehere19
maybehere02  maybehere05  maybehere08  maybehere11  maybehere14  maybehere17
bandit5@bandit:~/inhere$ find . -type f -size 1033c ! -executable
./maybehere07/.file2
bandit5@bandit:~/inhere$ cat ./maybehere07/.file2
pXa26xhMWaC2SvDotA4r9EgZku10eSBW
```

Fig 6 — Using find with size and permission filters to locate one specific file among dozens.

Lesson learned:

"find" is the right tool whenever you need to search by criteria (size, name, permissions, owner) rather than by browsing manually — it scales to any number of files or folders.

Level 6 → Level 7

Goal: The password file for bandit7 exists somewhere on the entire filesystem, owned by user bandit7 and group bandit6, and is exactly 33 bytes.

Commands used:

```
find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null  
cat /var/lib/dpkg/info/bandit7.password
```

This time the search starts from / (the root of the whole filesystem), so find inevitably tries to look inside folders the current user isn't allowed to open, which prints a wall of "Permission denied" errors. Redirecting those errors to /dev/null (a special file that discards anything written to it) keeps the output clean so only the real match shows up.

A terminal window with a dark background. At the top, it says "Enjoy your stay!". Below that, the user 'bandit6' at host 'bandit' runs the command 'find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null /var/lib/dpkg/info/bandit7.password'. The output shows the file path. Then, the user runs 'cat /var/lib/dpkg/info/bandit7.password' and the output is the password 'Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3'.

```
Enjoy your stay!  
bandit6@bandit:~$ find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null  
/var/lib/dpkg/info/bandit7.password  
bandit6@bandit:~$ cat /var/lib/dpkg/info/bandit7.password  
Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3  
bandit6@bandit:~$ |
```

Fig 7 — Searching the whole filesystem by owner, group and size, while silencing permission-denied noise.

Lesson learned:

2>/dev/null is a very common pattern: it hides error messages (file descriptor 2) so you can focus on the results that actually matter.

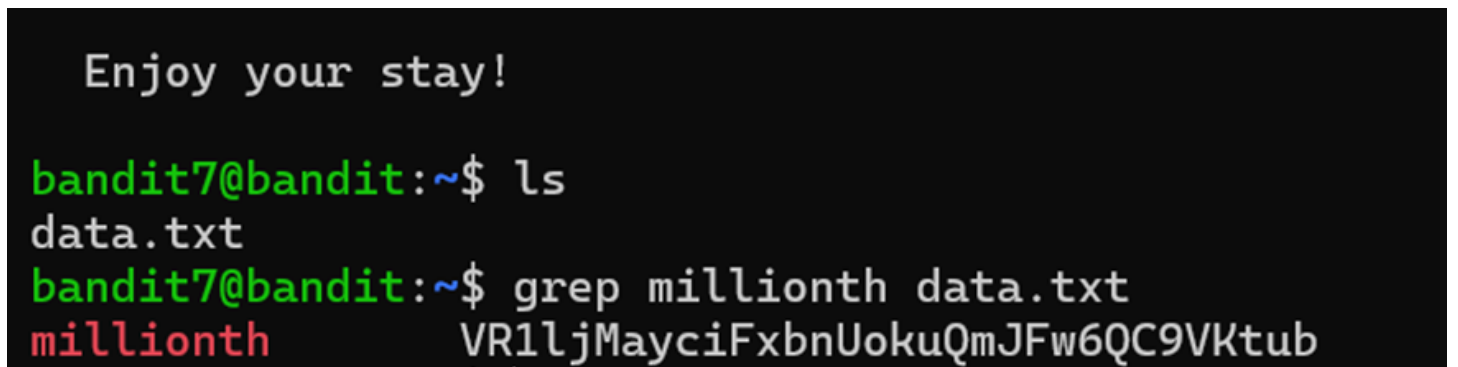
Level 7 → Level 8

Goal: The password is in data.txt, next to the word "millionth"

Commands used:

```
ls  
grep millionth data.txt
```

data.txt contains thousands of lines of random word/value pairs. "grep" searches text for a pattern and prints only the matching line(s) — so instead of scrolling through the whole file, `grep millionth data.txt` jumped straight to the one line containing that word, and the password sat right next to it.

A terminal window with a black background and white text. At the top, it says "Enjoy your stay!". Below that, the prompt "bandit7@bandit:~\$" is followed by the command "ls", which outputs "data.txt". Then, the prompt is followed by "grep millionth data.txt", which outputs "millionth" on the first line and "VR1ljMayciFxbnUokuQmJFw6QC9VKtub" on the second line.

```
Enjoy your stay!  
  
bandit7@bandit:~$ ls  
data.txt  
bandit7@bandit:~$ grep millionth data.txt  
millionth          VR1ljMayciFxbnUokuQmJFw6QC9VKtub
```

Fig 8 — Using grep to jump directly to the line containing a known keyword.

Lesson learned:

grep is the fastest way to find a known word or pattern inside a large file — no need to read the whole thing.

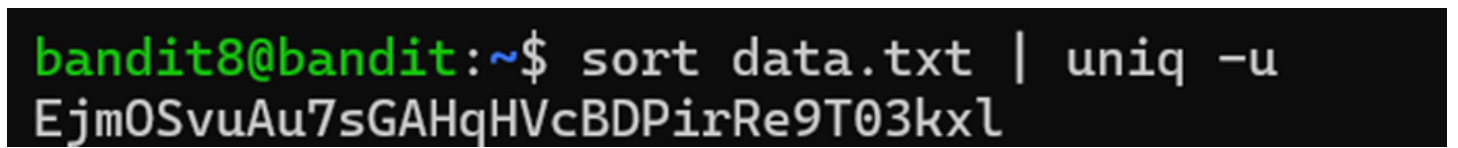
Level 8 → Level 9

Goal: data.txt has many lines, all duplicated except for one. Find the line that appears exactly once.

Commands used:

```
sort data.txt | uniq -u
```

"uniq" can only detect duplicate lines that sit next to each other, so the file has to be sorted first — that's what the pipe (|) does: it feeds sort's output straight into uniq as input. The -u flag tells uniq to print only lines that have no duplicates at all, which isolates the one unique password line instantly.

A terminal window with a black background. The prompt 'bandit8@bandit:~\$' is in green. The command 'sort data.txt | uniq -u' is entered in white. The output 'EjmOSvuAu7sGAHqHVcBDPirRe9T03kx1' is displayed in white on the next line.

```
bandit8@bandit:~$ sort data.txt | uniq -u  
EjmOSvuAu7sGAHqHVcBDPirRe9T03kx1
```

Fig 9 — Piping sort into uniq -u to isolate the one line with no duplicate.

Lesson learned:

The pipe (|) lets you chain simple tools together to solve something none of them could do alone — sort + uniq is a classic combo for finding duplicates or unique entries.

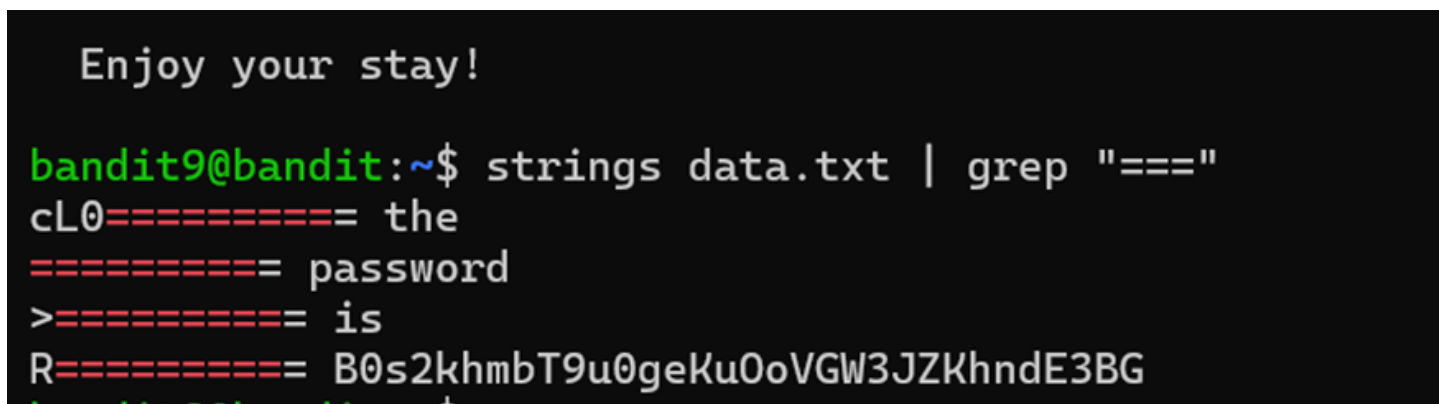
Level 9 → Level 10

Goal: data.txt is mostly binary garbage, but it contains one line of readable text starting with several '=' characters, followed by the password.

Commands used:

```
strings data.txt | grep "==="
```

Because the file has non-printable binary bytes mixed in, cat or grep alone on the raw file gets messy. "strings" pulls out only the human-readable text sequences from a binary file. Piping that into grep for a run of "=" characters (a pattern the challenge tells you to look for) filtered it down to the one line that mattered.



```
Enjoy your stay!

bandit9@bandit:~$ strings data.txt | grep "==="
cL0===== the
===== password
>===== is
R===== B0s2khmbT9u0geKu0oVGW3JZKhndE3BG
```

Fig 10 – Extracting readable text from a binary file with strings, then filtering with grep.

Lesson learned:

When a file is a mix of binary and text, "strings" is the tool to pull the readable parts out cleanly.

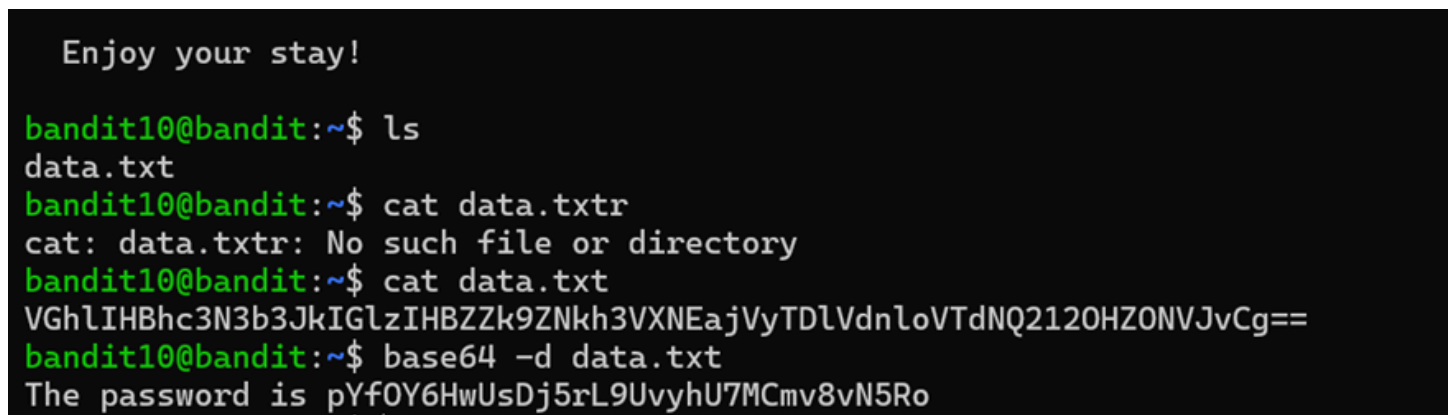
Level 10 → Level 11

Goal: data.txt contains the password encoded in Base64. Decode it.

Commands used:

```
ls
cat data.txtr # typo — command failed
cat data.txt
base64 -d data.txt
```

Base64 is a common way to represent binary data as plain text using only letters, numbers, +, / and =. Once you recognize that padding (the trailing ==) and character set, base64 -d (decode) converts it straight back to the original readable password.

A terminal window with a dark background and light-colored text. The prompt is 'bandit10@bandit:~\$'. The user enters 'ls' and the output is 'data.txt'. Then the user enters 'cat data.txtr' and gets an error: 'cat: data.txtr: No such file or directory'. Next, the user enters 'cat data.txt' and gets a long Base64 encoded string: 'VGhlIHBhc3N3b3JkIGlzIHBZZk9ZNkh3VXNEajVyTDlVdnloVTdNQ212OHZONVJvCg=='. Finally, the user enters 'base64 -d data.txt' and the output is 'The password is pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro'.

```
Enjoy your stay!

bandit10@bandit:~$ ls
data.txt
bandit10@bandit:~$ cat data.txtr
cat: data.txtr: No such file or directory
bandit10@bandit:~$ cat data.txt
VGhlIHBhc3N3b3JkIGlzIHBZZk9ZNkh3VXNEajVyTDlVdnloVTdNQ212OHZONVJvCg==
bandit10@bandit:~$ base64 -d data.txt
The password is pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro
```

Fig 11 —A typo (data.txtr) failed first; the correct filename then revealed the Base64 string, decoded with base64 -d.

Mistake I made: I mistyped the filename as "data.txtr" and got a "No such file or directory" error before catching my own typo.

Lesson learned:

Simple typos are one of the most common sources of errors on the command line — when you get "No such file or directory," the first thing to check is your spelling, not the system.

Level 11→ Level 12

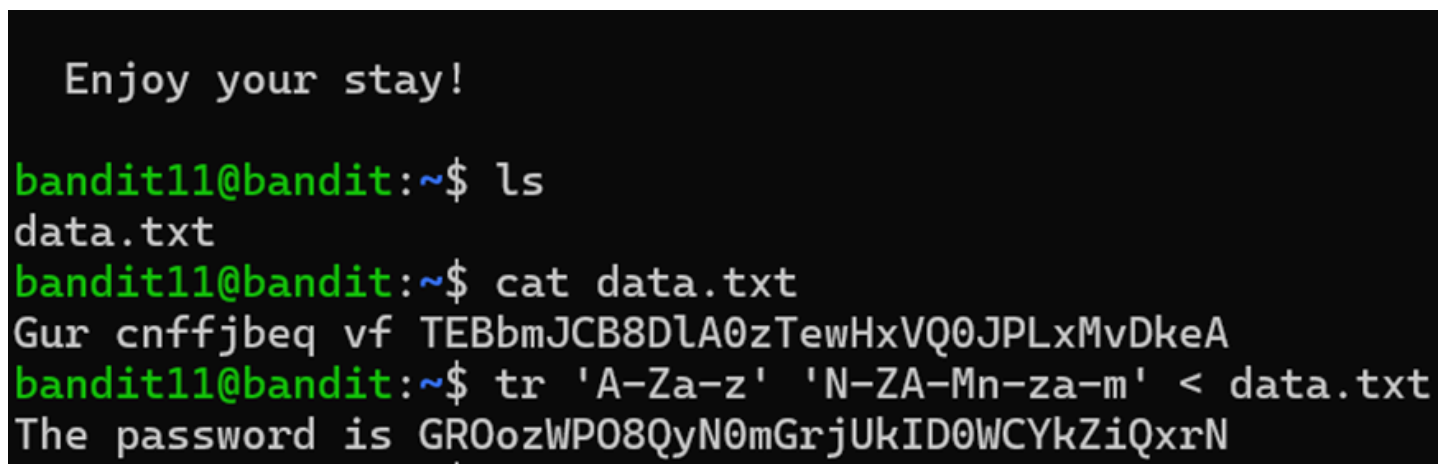
Goal: data.txt contains the password encrypted with ROT13 (each letter shifted 13 places through the alphabet).

Commands used:

```
cat data.txt
```

```
tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt
```

ROT13 is a simple substitution cipher — since the alphabet has 26 letters, shifting by 13 twice gets you back to the original text. "tr" translates one set of characters into another; the mapping 'A-Za-z' → 'N-ZA-Mn-za-m' shifts every letter by exactly 13 places, which both encodes and decodes ROT13.

A terminal window with a black background and green text. The prompt is 'bandit11@bandit:~\$'. The user enters 'ls' and the output is 'data.txt'. Then the user enters 'cat data.txt' and the output is 'Gur cnffjbeq vf TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA'. Finally, the user enters 'tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt' and the output is 'The password is GR00zWP08QyN0mGrjUKID0WCYkZiQxrN'.

```
Enjoy your stay!  
  
bandit11@bandit:~$ ls  
data.txt  
bandit11@bandit:~$ cat data.txt  
Gur cnffjbeq vf TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA  
bandit11@bandit:~$ tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt  
The password is GR00zWP08QyN0mGrjUKID0WCYkZiQxrN
```

Fig 12 — Decoding a ROT13-encoded string using tr.

Lesson learned:

ROT13 is easy to spot: the scrambled text still 'looks' like normal words with the same length and spacing, just with letters swapped — a strong hint that a simple substitution cipher is involved.

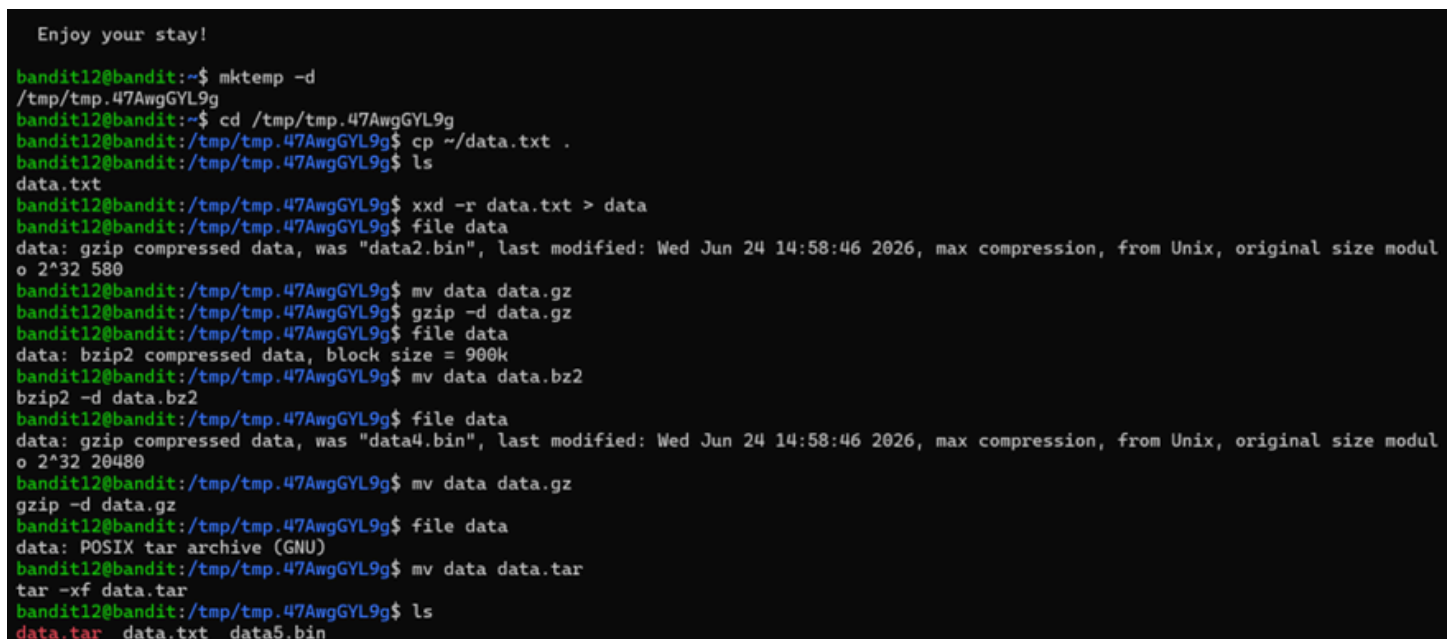
Level 12 → Level 13

Goal: ~/data.txt is a hexdump of a file that has been compressed multiple times, using different compression methods stacked on top of each other. Recover the original password.

Commands used:

```
mkdir /tmp/tmp.XXXX && cd into it (I used: mktemp -d, then cd)
cp ~/data.txt .
xxd -r data.txt > data # reverse the hex dump back into real binary
file data # check what type it actually is
mv data data.gz && gzip -d data.gz
file data # -> bzip2
mv data data.bz2 && bzip2 -d data.bz2
file data # -> gzip again
...repeat file -> rename -> decompress...
tar -xf data.tar
cat data8 # finally plain text
```

This is the most involved level in this batch. The file starts life as a hex dump (a text representation of raw bytes), so the first step is `xxd -r` to turn it back into real binary data. After that, it's a loop: run "file" to identify the current compression format, rename the file with the matching extension (`mv`), decompress it with the right tool (`gzip`, `bzip2`, or `tar`), and repeat — because the challenge deliberately nests several compression layers (`gzip` → `bzip2` → `gzip` → `tar` → `bzip2` → `gzip` → `tar` → `gzip`) inside one another. I worked entirely inside a throwaway temp directory (made with `mktemp -d`) so I wouldn't clutter my home folder.



```
Enjoy your stay!
bandit12@bandit:~$ mktemp -d
/tmp/tmp.47AwgGYL9g
bandit12@bandit:~$ cd /tmp/tmp.47AwgGYL9g
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ cp ~/data.txt .
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ ls
data.txt
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ xxd -r data.txt > data
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file data
data: gzip compressed data, was "data2.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modul
o 2^32 580
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data data.gz
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ gzip -d data.gz
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file data
data: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data data.bz2
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ bzip2 -d data.bz2
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file data
data: gzip compressed data, was "data4.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modul
o 2^32 20480
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data data.gz
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ gzip -d data.gz
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file data
data: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data data.tar
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ tar -xf data.tar
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ ls
data.tar  data.txt  data5.bin
```

Fig 13a — First few rounds: hex-decoding, then peeling off `gzip`, `bzip2`, and `tar` layers one at a time.

```

bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file *
data5.bin: POSIX tar archive (GNU)
data.tar:  POSIX tar archive (GNU)
data.txt:  ASCII text
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data5.bin data5.tar
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ tar -xf data5.tar
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ ls
data.tar  data.txt  data5.tar  data6.bin
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file *
data5.tar: POSIX tar archive (GNU)
data6.bin: bzip2 compressed data, block size = 900k
data.tar:  POSIX tar archive (GNU)
data.txt:  ASCII text
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data6.bin data6.bz2
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ bzip2 -d data6.bz2
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file *
data5.tar: POSIX tar archive (GNU)
data6:     POSIX tar archive (GNU)
data.tar:  POSIX tar archive (GNU)
data.txt:  ASCII text
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ tar -xf data6
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file *
data5.tar: POSIX tar archive (GNU)
data6:     POSIX tar archive (GNU)
data8.bin: gzip compressed data, was "data9.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size
modulo 2^32 49
data.tar:  POSIX tar archive (GNU)
data.txt:  ASCII text
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ mv data8.bin data8.gz
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ gzip -d data8.gz

```

Fig 13b— Continuing to identify and strip away each remaining compression layer

```

bandit12@bandit:/tmp/tmp.47AwgGYL9g$ file *
data5.tar: POSIX tar archive (GNU)
data6:     POSIX tar archive (GNU)
data8:     ASCII text
data.tar:  POSIX tar archive (GNU)
data.txt:  ASCII text
bandit12@bandit:/tmp/tmp.47AwgGYL9g$ cat data8
The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk

```

Fig 13c— The final layer decompresses to a plain-text file containing the password.

Lesson learned:

Never assume a file's extension (or lack of one) tells you what it is. The "file → rename → decompress → repeat" loop is a reusable pattern for any multi-layered/obfuscated compressed file.

Level 13 → Level 14

Goal: bandit13's home directory contains a private SSH key (sshkey.private) that logs in directly as bandit14 — no password needed.

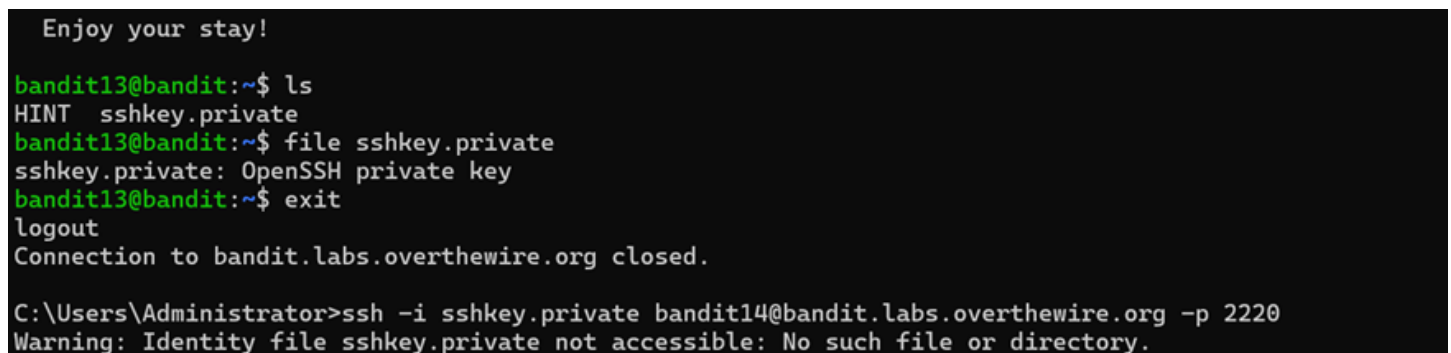
Commands used:

```
ls # HINT sshkey.private
```

```
file sshkey.private # confirms: OpenSSH private key
```

```
ssh -i sshkey.private bandit14@bandit.labs.overthewire.org -p 2220
```

The -i flag tells ssh "use this specific private key file to authenticate" instead of asking for a password. In theory this alone should be enough — but I ran into several real-world snags getting there, shown below.



```
Enjoy your stay!
bandit13@bandit:~$ ls
HINT  sshkey.private
bandit13@bandit:~$ file sshkey.private
sshkey.private: OpenSSH private key
bandit13@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

C:\Users\Administrator>ssh -i sshkey.private bandit14@bandit.labs.overthewire.org -p 2220
Warning: Identity file sshkey.private not accessible: No such file or directory.
```

Fig 14a — Mistake: I copied the key path wrong / tried to use it from the wrong machine, and ssh couldn't find the key file at all.

Mistake I made: My first `ssh -i sshkey.private ...` attempt failed with "Identity file sshkey.private not accessible: No such file or directory" — I was trying to reference the key from my local Windows machine before actually copying it there.

```

Enjoy your stay!

bandit13@bandit:~$ ssh -i /tmp/bandit14_key bandit14@bandit.labs.overthewire.org -p 2220
The authenticity of host '[bandit.labs.overthewire.org]:2220 ([127.0.0.1]:2220)' can't be established.
ED25519 key fingerprint is: SHA256:C2ihUBV7ihnV1wUXRb4RrEcLfXC5CXlhmAAM/urerLY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Could not create directory '/home/bandit13/.ssh' (Permission denied).
Failed to add the host to the list of known hosts (/home/bandit13/.ssh/known_hosts).

      _ _ _ _ _
     | | | | |
    _|_|_|_|_|_
   | | | | | | |
  _|_|_|_|_|_|_
 | | | | | | |
_|_|_|_|_|_|_|

This is an OverTheWire game server.
More information on http://www.overthewire.org/wargames

!!! You are trying to log into this SSH server with a password on port 2220 from localhost.
!!! Connecting from localhost is blocked to conserve resources.
!!! Please log out and log in again.

backend: gibson-1
Received disconnect from 127.0.0.1 port 2220:2: no authentication methods enabled
Disconnected from 127.0.0.1 port 2220
bandit13@bandit:~$ whoami
bandit13
bandit13@bandit:~$ cat /etc/bandit_pass/bandit14
cat: /etc/bandit_pass/bandit14: Permission denied
bandit13@bandit:~$ ssh -i sshkey.private bandit14@localhost

```

Fig 14b — Mistake: after copying the key locally, I also tried connecting from localhost on the wrong port, and the server actively blocks password-based connections from localhost to save resources.

Mistake I made: I tried `ssh -i sshkey.private bandit14@localhost` from inside the bandit13 session (on the default port 22). OverTheWire's server explicitly rejects this: it's not intended to be used that way, and it also blocks localhost logins with a password for resource reasons.

```

C:\Users\Administrator>icacls C:\Users\Administrator\sshkey.private /inheritance:r
processed file: C:\Users\Administrator\sshkey.private
Successfully processed 1 files; Failed processing 0 files

C:\Users\Administrator>icacls C:\Users\Administrator\sshkey.private /grant:r "%USERNAME%:R"
processed file: C:\Users\Administrator\sshkey.private
Successfully processed 1 files; Failed processing 0 files

C:\Users\Administrator>ssh -i C:\Users\Administrator\sshkey.private bandit14@bandit.labs.overthewire.org -p 2220

```

Fig 14c — The actual fix: on Windows, SSH refuses to use a private key file that has overly loose permissions, so I used `icacls` to lock the key down to just my own user before it would be accepted.

SSH is strict about private key file permissions — if a key is readable by "too many" users, most SSH clients (including Windows OpenSSH) will refuse to use it as a security precaution. On Linux this is normally fixed with `chmod 600 keyfile`. On Windows, the equivalent is `icacls`, which resets the file's permissions (`/inheritance:r` removes inherited permissions, `/grant:r "%USERNAME%:R"` grants read-only access to just the current user).

Enjoy your stay!

```
bandit14@bandit:~$ whoami  
bandit14  
bandit14@bandit:~$ |
```

Fig 14d – Success: whoami confirms I'm logged in as bandit14, straight from the private key with no password.

Lesson learned:

(1) Always double-check where a file actually lives before referencing it in a command. (2) The intended way to reach the next level is over the real SSH endpoint (bandit.labs.overthewire.org -p 2220), not localhost. (3) A private key's file permissions matter just as much as the key itself – lock it down (chmod 600 on Linux, icacis on Windows) or SSH will refuse to use it.

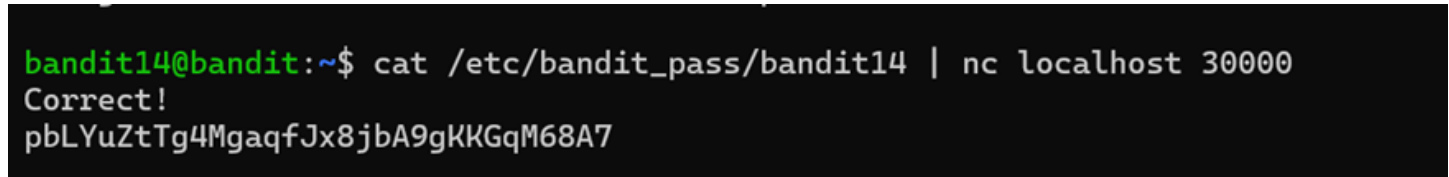
Level 14 → Level 15

Goal: Submit bandit14's own password to local port 30000 to receive bandit15's password.

Commands used:

```
cat /etc/bandit_pass/bandit14 | nc localhost 30000
```

There's a small custom program listening on port 30000 that checks whatever text it receives against the correct password for bandit14. "nc" (netcat) is a simple tool for sending/receiving raw data over a network connection. Piping the password straight into nc localhost 30000 sends it to that listening service, which replies with "Correct!" followed by the next password.



```
bandit14@bandit:~$ cat /etc/bandit_pass/bandit14 | nc localhost 30000
Correct!
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
```

Fig 15 — Piping the current level's password into netcat to "talk" directly to a service on port 30000.

Lesson learned:

netcat is often called the "Swiss army knife" of networking — it's a simple, direct way to send data to or read data from any open port.

Level 15 → Level 16

Goal: Submit bandit15's password to local port 30001 — but this time the port expects an encrypted (SSL/TLS) connection, not plain text.

Commands used:

```
openssl s_client -connect localhost:30001
```

```
# then, once connected, paste the password and press Enter
```

Plain netcat can't speak SSL/TLS, so connecting to this port directly would fail or hang. openssl s_client acts like netcat but wraps the connection in SSL first — performing the handshake, checking (and, in this case, accepting) the server's self-signed certificate — and then hands you an interactive session where you can type the password directly. The service replies "Correct!" followed by bandit16's password.

```
Enjoy your stay!

bandit15@bandit:~$ openssl s_client -connect localhost:30001
Connecting to 127.0.0.1
CONNECTED(000000003)
Can't use SSL_get_servername
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=SnakeOil
verify return:1
---
Certificate chain
 0 s:CN=SnakeOil
  i:CN=SnakeOil
  a:PKEY: RSA, 4096 (bit); sigalg: sha256WithRSAEncryption
  v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun  8 03:59:50 2034 GMT
```

Fig 16a — Opening an SSL/TLS connection to the service with openssl s_client, including its self-signed certificate details.

```

Session-ID: ABA975FEE03A3DF7DEF88E30015E0A9E6F1969A57087063F9D0D7FBD7ED8925A
Session-ID-ctx:
Resumption PSK: ED2421506FA9277142342E5BFA0C25FF479746B07F7A4FC11A286F5020DBAC6DB59EE3F8C9B57D7CD7628DB622355D8A
PSK identity: None
PSK identity hint: None
SRP username: None
TLS session ticket lifetime hint: 300 (seconds)
TLS session ticket:
0000 - 67 e3 14 59 05 ac c1 03-b5 75 90 9c 77 85 f9 8e g..Y.....u..w...
0010 - f3 53 b1 5f 1e 5f 9d e9-22 58 4d e3 83 56 f0 b5 .S._..."XM..V...
0020 - 54 35 a0 5d 86 ce c5 9f-9c 1a 29 ca 86 a6 b2 48 T5.].....)....H
0030 - 63 6a 11 28 39 79 3b db-1a 6e 22 f9 cc ad 5c 11 cj.(9y;..n"....\..
0040 - b7 88 cf d6 4c c5 86 7b-7b 01 da 99 a2 d1 f6 34 ....L..{{.....4
0050 - 8a 6c ce b4 30 92 1c c9-41 d8 68 7d 96 ac 81 c5 .l..0...A.h}....
0060 - 40 78 28 fb 85 c7 de 1c-5e 68 c1 54 9d 07 06 37 @x(.....^h.T...7
0070 - 30 8c 2f 20 1f 73 95 e2-2d 4a bc 69 3a c1 55 11 0./ .s..-J.i:.U.
0080 - ae 30 ca dc f4 fd 04 92-b2 50 99 3a dc 2e f2 fc .0.....P:.....
0090 - cc e8 8b 56 7e 19 ef 65-c9 84 e8 b4 6e f1 fe 8f ...V~.e....n...
00a0 - 64 2e 24 8e 0a 56 2a 36-f6 ed 02 3b 23 ce c5 cd d.$..V*6...;#...
00b0 - fd a7 ea 66 73 85 5f 57-36 e7 9b 63 6a 4d 30 41 ...fs._W6..cjm0A
00c0 - 48 9c 5e 8f 21 ba f7 a8-1c 28 21 0e 75 6c 5e c4 H.^!....(!.ul^..
00d0 - 0f 1f 67 c8 80 d2 97 73-bc 38 c0 c2 a1 60 af 2c ..g....s.8...'.,

Start Time: 1789009173
Timeout : 7200 (sec)
Verify return code: 18 (self-signed certificate)
Extended master secret: no
Max Early Data: 0
---
read R BLOCK
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
Correct!
kS0Hf0u5HiXFwKMKFqXvPd0TNGGa0X8V

```

Fig 16b – After the handshake completes, typing in bandit15's password returns "Correct!" and the password for bandit16.

Lesson learned:

Not every network service speaks plain text – some expect an encrypted channel. openssl s_client is the go-to tool for manually talking to an SSL/TLS service the same way netcat talks to a plain one.

Common Mistakes & Lessons — Quick Reference

If you're just starting Bandit, here are the mistakes I actually made, condensed into one list so you can watch out for them:

- Typing a filename wrong (e.g. `data.txtr` instead of `data.txt`) — always re-read the exact filename shown by `ls` before using it.
- Trying to use a file (like an SSH key) before it's actually been copied to the machine you're running the command from.
- Confusing port 22 (default SSH) with port 2220 (the port OverTheWire actually uses) — always double check the port in your `ssh -p` flag.
- Forgetting that private SSH keys need strict, locked-down file permissions (`chmod 600` on Linux, `icacls` on Windows) before SSH will accept them.
- Trying to log in through `localhost` when the exercise expects the real remote hostname (and vice versa).
- Forgetting to quote or escape filenames containing spaces, dashes, or other special characters.

General Tips for Beginners

- Run `"ls"` and `"ls -a"` first in every new level — don't guess, look.
- When a command "hangs" with no output, it's very often waiting on `stdin` (as with `cat -`). `Ctrl+C` to cancel and figure out why.
- Use `"file"` to check what a mystery file actually is before assuming its type from its name or extension.
- Read error messages fully — "Permission denied", "No such file or directory", and "not accessible" all point to a different kind of fix.
- It's completely normal to fail a command two or three times before it works. The screenshots in this document are proof of that, even at a level that was eventually solved.

Conclusion

Working through Bandit levels 0–15 covered a huge range of practical Linux and basic networking/security skills: navigating hidden and oddly-named files, filtering and searching text (`grep`, `strings`, `sort/uniq`), understanding file permissions, working with SSH keys, and talking directly to network services with `netcat` and `openssl`. The mistakes along the way were just as valuable as the successes — they're the reason I now recognize these errors instantly instead of getting stuck on them. I hope this write-up helps someone else get past the exact same walls I hit.